

SDV Dashboard: A Lightweight Backend-Free Web-Based Dashboard for ROS 2 Robot Monitoring, Visualization, and Control

Tholuchuru Sahana

Department of CSE
GITAM Deemed to be University
Bengaluru, India.
tholuchurusahana2005@gmail.com

GC Harsha Vardhan Reddy

Department of EECE
GITAM Deemed to be University
Bengaluru, India.
harshavardhanh426@gmail.com

Venkata Kranthi B

Dept of EECE
GITAM Deemed to be university
Bengaluru, India
kbudigi@gitam.edu

Abstract--Web-based dashboards for the Robot Operating System 2 (ROS 2) typically require either a dedicated visualization application, a custom backend service, or both, which increases setup effort for small-scale and educational robotics projects. This paper presents SDV Dashboard, a single-file, browser-based monitoring and teleoperation interface for ROS 2 mobile robots. The dashboard is client-side backend-free (a serverless frontend): it is implemented entirely in HTML, CSS, and JavaScript and requires no build step, package installation, or custom backend code of its own; it connects directly to a robot's existing `rosbridge_suite` WebSocket server using the `roslibjs` client library. The interface composes the map-to-odom and odom-to-base transforms from the `/tf` topic to estimate robot pose, renders an occupancy grid, costmap overlay, LiDAR scan, and planned paths on an HTML5 canvas, and provides keyboard- and joystick-based teleoperation alongside click-to-set Nav2 goal publishing. Camera video is served separately through an HTTP MJPEG stream, consistent with the `web_video_server` convention, so the complete operator-side workflow depends on two lightweight services running on the robot rather than on any software installed on the client. We describe the system architecture, implementation, and feature set, and position the dashboard relative to existing ROS 2 web and desktop visualization tools, including Foxglove, Webviz, ROSboard, Robot Web Tools components, and RViz2. We additionally report an informal quantitative evaluation -- connection reliability, message latency, render rate, browser CPU and memory usage, network bandwidth, teleoperation responsiveness, and a scalability sweep across map size and LiDAR model -- obtained through manual timing and built-in browser and OS tools rather than controlled, instrumented benchmarking; a rigorous, same-machine comparison against the surveyed tools remains a direction for future work.

Index Terms--ROS 2, `rosbridge`, `roslibjs`, web-based robotics, teleoperation, robot monitoring dashboard, Nav2, WebSocket, serverless frontend, human-robot interaction

I. INTRODUCTION

ROS 2 separates robotics software into modular nodes communicating over topics, services, and actions [1], but this modularity means that monitoring or controlling a robot typically requires a client that speaks the ROS 2 protocol.

The most established option, RViz2, is a powerful 3D visualization tool, but as a GPU-accelerated desktop application with a plugin-based architecture [2], [3].

It is a poor fit for quick access from a phone, a lab kiosk, or any machine without a full ROS 2 and GPU install.

The Robot Web Tools project addressed this gap with `rosbridge_suite`, exposing ROS topics, services, and parameters over a JSON-over-WebSocket protocol, and `roslibjs`, a JavaScript client consuming this protocol in any browser [4]–[6]. This pairing enabled Webviz [7], its successor Foxglove [8], [9], and ROSboard [10] to provide browser-accessible visualization without a local ROS 2 install. However, several of these tools depend on a dedicated server process, as with ROSboard's Tornado-based node [10], or have moved toward account-gated, commercially licensed platforms, as with Foxglove's discontinuation of open-source Studio in favor of a paid Data Platform [9], [11] -- both adding friction for student and hobbyist projects that need a fast, no-account way to operate a single robot.

This paper presents SDV Dashboard, a monitoring and teleoperation interface built as a single static HTML file with no backend code, build process, or package manager of its own, opened directly in a browser and connected to a robot's existing `rosbridge_suite` endpoint. Once connected, it subscribes to odometry, transform, LiDAR, occupancy grid, costmap, and path topics to render a live 2D map, and publishes `geometry_msgs/Twist` and `geometry_msgs/PoseStamped` messages for teleoperation and Nav2 goal-setting. A separate panel shows a live camera feed over HTTP as an MJPEG stream, following the `web_video_server` convention.

The contribution of this paper is a description of the dashboard's architecture, implementation, and feature set; a structured comparison against existing ROS 2 visualization tools along architecture, deployment, and control axes; and an informal quantitative evaluation of connection reliability, latency, rendering performance, resource usage, and bandwidth, obtained with manual timing and built-in tools rather than dedicated instrumented logging. A controlled, same-machine comparison against RViz2, Foxglove, and ROSboard,

together with automated logging and a multi-camera scalability test, is identified as future work.

Section II reviews related tools. Section III describes the architecture. Section IV details the internal design. Section V discusses safety and security. Section VI describes the implementation. Section VII lists the feature set. Sections VIII-IX describe the evaluation. Section X compares the dashboard with existing tools.

II. RELATED WORK

This section reviews six categories of tooling relevant to ROS 2 monitoring and control, drawing only on cited official documentation or peer-reviewed publications.

A. RViz2

RViz2, the ROS 2 port of RViz, is the de facto standard for visualizing sensor data, robot models, and costmaps through a plugin-based display architecture [2], [3]. As a GPU-accelerated desktop application requiring a full ROS 2 install, it is poorly suited to embedded or resource-constrained platforms, and supports control only indirectly through panels such as Nav2 goal-setting.

B. *rosbridge_suite* and Robot Web Tools

rosbridge_suite lets any JSON-capable client publish, subscribe, and call services over WebSockets without a native ROS client library [4], [5]; *roslibjs* implements this protocol for browsers [6], and the server listens on port 9090 by default [5]. Crick et al. introduced the concept to make ROS accessible to non-ROS and HRI applications [4]. Since it exposes message types generically, visualization and control are built on top of it by clients such as Webviz, Foxglove's legacy connector, and this dashboard.

C. Webviz

Webviz is an open-source, panel-based browser application built by Cruise Automation for visualizing live data and ROS bag files, connecting by default over `ws://localhost:9090` with all processing performed client-side [7]. It is no longer commercially supported, and its documentation recommends Foxglove -- a fork by former Cruise engineers -- for new users [7].

D. Foxglove

Foxglove is a visualization and data-management platform available as a desktop and browser client [8]. For ROS 2 it now recommends its purpose-built *foxglove_bridge* over *rosbridge_suite* for performance and stability, though *rosbridge* connectivity remains supported [8]. With Foxglove 2.0, the open-source Studio edition was discontinued in favor of a commercially licensed, tiered platform [9], [11].

E. ROSboard

ROSboard runs as a ROS node on the robot and serves a web interface over a custom Tornado-based WebSocket rather than *rosbridge_suite*, a deliberate choice its author attributes to easier ROS1/ROS2 cross-compatibility, lossy compression of large messages, and limiting client access to the ROS graph [10]. It is lightweight and mobile-friendly [12], but unlike this

dashboard, requires installing a dedicated Python package on the robot.

F. Browser-Native ROS 2 Environments

A more recent direction compiles ROS 2 itself to WebAssembly so nodes run entirely in the browser sandbox. ROS2WASM, built on the RoboStack distribution, also supports connecting to external physical robots via *roslibjs* [13]. This differs philosophically from SDV Dashboard, which remains a thin client depending entirely on an external, already-running *rosbridge* endpoint.

G. Summary

Tools minimizing client-side installation (Webviz, Foxglove's *rosbridge* mode, this dashboard) depend on *rosbridge_suite*, while tools minimizing robot-side dependency (ROSboard, *foxglove_bridge*) introduce a custom bridge instead; RViz2 occupies a third position, requiring a full local, GPU-capable install. SDV Dashboard sits in the first category, prioritizing zero client-side installation at the cost of depending on *rosbridge_suite* and *web_video_server*.

III. SYSTEM ARCHITECTURE

SDV Dashboard follows a thin-client architecture: all application logic runs inside the operator's browser, while all robot-side functionality is provided by two standard, separately maintained services rather than custom code -- *rosbridge_suite*, exposing the ROS 2 graph over a WebSocket using the *rosbridge* v2.0 JSON protocol [4], [5], and a *web_video_server*-style HTTP MJPEG stream. The dashboard itself contributes no server-side software: a single HTML file that a browser loads directly, from disk or any static host.

Fig. 1 depicts the resulting data flow. On the robot, *rosbridge_server* listens on TCP port 9090, the *rosbridge_suite* default [5]. The browser opens a *roslib.Ros* WebSocket connection to `ws://<robot-ip>:9090`, where `<robot-ip>` is entered by the operator. Once the handshake completes, the dashboard subscribes to a fixed set of topics (Section IV) and creates two publisher handles for teleoperation and navigation goals. Independently, the camera panel issues an HTTP GET to `http://<robot-ip>:8080/stream`, answered with a continuous multipart MJPEG stream rendered as an `<img>` element.

This two-service arrangement means the system is not server-side-serverless in an absolute sense, since robot-side services are required; 'backend-free' here describes specifically the client side, which needs no application server, database, build pipeline, or install step of its own, and can be opened directly from a filesystem path or plain static file host.



Fig. 1. Overall SDV Dashboard Architecture: the map/LiDAR view (left), camera feed (upper right) showing the MJPEG stream, and teleoperation panel (lower right) with WASD/joystick control and emergency-stop, connected to the robot's `rosbridge_suite` server independently of the camera's HTTP connection

IV. SYSTEM DESIGN

The dashboard's internal design separates into four concerns: connection management, ROS interface setup, pose estimation, and rendering.

A. Connection Management

A single global `ROSLIB.Ros` instance is created when the operator presses Connect. The dashboard listens for the connection, error, and close lifecycle events exposed by `roslibjs`, updating a status indicator, and on a successful connection tears down and recreates its topic subscriptions, avoiding duplicates when reconnecting to a different robot IP during a session.

B. ROS Interface Setup

Upon connection, the dashboard subscribes to `/odom` (fallback pose before transform data is available), `/tf`, `/scan`, an operator-configurable occupancy grid topic defaulting to `/map`, `/global_costmap/costmap`, `/plan`, and `/local_plan`, and creates two publishers: `/cmd_vel` for teleoperation and `/goal_pose` for Nav2 goal requests. This topic set corresponds to a standard Nav2-enabled differential-drive or skid-steer configuration; topics outside this set are not exposed.

C. Pose Estimation

Rather than subscribing to a single pre-fused pose topic, the dashboard reconstructs robot pose in the map frame by composing two transforms from `/tf`: map-to-odom and odom-

to-base. Each incoming `TFMessage` is inspected for matching frame pairs, quaternion orientation is converted to yaw, and the transforms are cached; pose is then the planar composition of both -- the standard method for resolving map-frame pose when no single combined transform is published [14]. Before any map-to-odom transform is observed, the dashboard falls back to treating `/odom` position directly as an approximate pose, adequate for short windows but without loop-closure correction.

D. Rendering Pipeline

All map-panel output is drawn on a single HTML5 canvas, redrawn through `requestAnimationFrame` on a schedule-and-coalesce pattern: incoming data marks a redraw pending, and at most one redraw occurs per animation frame regardless of message volume. Occupancy grid and costmap messages are rasterized once into an offscreen canvas on receipt, then drawn via `drawImage` with scale and rotation rather than re-rasterized every frame. The canvas renders, back-to-front, a reference grid, global and local paths, LiDAR points, an active goal marker, and a robot icon oriented to current yaw.

V. SAFETY AND SECURITY CONSIDERATIONS

The dashboard's dependence on standard, unmodified `rosbridge_suite` and `web_video_server` services carries safety and security implications worth stating explicitly. Both robot-side services are unencrypted by default -- `ws://` rather than `wss://`, and plain `http://` for video -- and neither channel is authenticated; the dashboard is therefore intended for a trusted local network (a lab, classroom, or private robot subnet), not

public or untrusted deployment, without an additional layer of encryption and access control that the dashboard does not itself provide.

If the WebSocket drops mid-teleoperation, the dashboard stops publishing, but whether the robot itself treats silence as a stop condition depends entirely on robot-side driver behavior outside the dashboard's control. We recommend, as a design consideration for robot-side integration rather than a feature implemented or tested here, that nodes subscribing to `/cmd_vel` implement a command timeout that zeroes velocity if no message arrives within a bounded interval (e.g., 500 ms). Section VIII reports an observed stop time following disconnection, but this was not verified to originate from any such timeout, and none is implemented in the dashboard itself. Reconnection avoids duplicate subscriptions (Section IV) and was exercised across a small number of connect/disconnect/reconnect cycles, but has not been systematically characterized under adverse network conditions or long-running sessions.

VI. IMPLEMENTATION

The dashboard is a single .html file containing inline CSS and JavaScript, with one external dependency loaded from a CDN: `roslib.js`, version 1.x [6]. No other libraries, frameworks, or build tools are used; there is no `package.json`, `bundler`, or transpilation step.

A. Frontend Structure

The interface is a CSS grid with three panels: a map/LiDAR view spanning the left column at full height, a camera feed panel upper right, and a teleoperation panel lower right. A persistent top bar exposes the robot IP field, configurable topic fields, a connection status indicator, a live pose readout, and an emergency-stop control. Two responsive breakpoints (1080 px and 600 px) collapse the grid into a single stacked column for tablets and phones.

B. Teleoperation Implementation

Two independent input methods publish to `/cmd_vel`: WASD keyboard control, combined into a linear/angular velocity pair scaled by adjustable speed sliders and republished every 100 ms while a key is held, and a draggable virtual joystick scaling the same sliders. Both are interrupted by an emergency-stop that clears all input state and publishes a zero-velocity Twist, additionally bound to the spacebar.

C. Navigation Goal Implementation

Double-clicking the map canvas converts pixel coordinates into map-frame coordinates using the panel's current pan and zoom, computes a target orientation pointing from the robot toward the click, and publishes a PoseStamped on `/goal_pose` matching the interface expected by Nav2's basic navigator. A separate control publishes a second, fixed goal pose returning the robot to a pre-configured home position, presently hard-coded in the client script.

D. Camera Implementation

The camera panel is a standard `<img>` element whose `src` is an MJPEG URL of the form `http://<robot-ip>:8080/stream?topic=<topic>&type=mjpeg`, matching the query convention used by `web_video_server`; the dashboard assumes a compatible server is already running and falls back to a placeholder graphic with a manual refresh control when the stream fails.

VII. FEATURES

The current implementation provides the following operator-facing capabilities, each directly traceable to Section VI:

- Live 2D map view combining occupancy grid, optional costmap overlay, planned paths, LiDAR points, and a pose-tracked robot icon, with zoom and pan.
- Optional camera-following mode re-centering the map on the robot's pose.
- Click-to-set Nav2 goal publishing, plus a home-goal shortcut and a goal-cancel control.
- Dual-mode teleoperation (WASD or virtual joystick) with adjustable speed limits and a live velocity readout.
- Dedicated emergency-stop, bound to a top-bar button and the spacebar.
- Live camera feed with an editable topic field and manual stream-refresh.
- Live telemetry overlay reporting position, heading, LiDAR point count, and grid dimensions.
- Editable robot IP and topic fields, letting one file target different robots without code changes.
- Responsive layout collapsing to a single column for tablets and phones.

All of the above are implemented and present in the reviewed source file; none is planned or partial.

VIII. EXPERIMENTAL SETUP

This section describes the deployment context and informal evaluation methodology used to characterize the dashboard's behavior in response to reviewer requests for quantitative data. The dashboard was evaluated against a ROS 2 stack exposing the standard Nav2 topic set (Section IV), connected via `rosbridge_suite` running on an onboard Jetson-class computer, over local WiFi to an operator laptop running the dashboard in a Chromium-based browser.

Eight informal tests were conducted: (1) connection and reliability across repeated connect/disconnect/reconnect cycles, (2) message latency, comparing robot-side publish rates against perceived end-to-end delay, (3) canvas render rate, (4) browser CPU and memory, (5) network bandwidth, (6) teleoperation responsiveness, (7) a scalability sweep across grid size and LiDAR model, and (8) a reference comparison of RViz2's resource usage on the robot's onboard compute. These were measured using manual timing, `ros2 topic hz`, Chrome's Task Manager and DevTools, and system-level network monitoring, rather than dedicated instrumented logging with large trial

counts; results are accordingly approximate, single-session observations rather than a statistically rigorous benchmark.

IX. RESULTS AND DISCUSSION

This section reports the results of the eight tests in Section VIII. Figures throughout are approximate and representative of the specific session and network conditions observed, not rigorous benchmark results.

A. Connection, Latency, and Rendering

Across ~ 10 connect/disconnect/reconnect trials, the dashboard connected in ~ 5.0 s on average (range ~ 4.5 – 5.5 s), and WASD teleoperation showed no perceptible latency. After an intentional WiFi disconnection during motion, the robot stopped within ~ 1.0 s, and control resumed within ~ 1.0 s of reconnection; emergency-stop halted motion in under 1 s throughout, though as noted in Section V this was not verified to originate from any explicit timeout. Robot-side publish rates, measured independently via `ros2 topic hz`, averaged ~ 10.2 Hz for `/scan` and ~ 190 Hz for `/tf`. Despite this, perceived end-to-end delay for LiDAR and camera visualization was ~ 3 – 4 s, while teleoperation, goal publishing, and emergency-stop remained immediate, consistent with lightweight outbound control versus bandwidth-heavy inbound traffic rather than a rendering-pipeline limitation. Canvas render rate, capped architecturally at 60 FPS, was observed at ~ 10 – 15 FPS, tracking the `/scan` arrival rate rather than the render loop's own capacity.

B. Resource Usage and Bandwidth

The browser tab reported ~ 89 MB memory (Chrome Task Manager; ~ 5.29 MB JS heap) and ~ 14 – 19% CPU with no clear difference between idle and active states, suggesting continuous `/tf` processing dominates cost over rendering or control logic. Over a 5-minute session with all data streams active, outbound traffic remained ~ 150 – 250 Kbps while inbound traffic (`/tf`, `/scan`, `map/costmap`, and MJPEG video) totaled ~ 10 – 35 MB, an asymmetry consistent with the dashboard's thin-client design and explaining the visualization delay above.

C. Teleoperation and Scalability

WASD and emergency-stop response remained under 1 s throughout, unaffected by the ~ 3 – 4 s visualization delay, confirming the control path is robust to network conditions that degrade visualization. A scalability sweep across occupancy grid size (small vs. large map) and LiDAR model (YDLIDAR X2 vs. TG30) showed no observable difference in rendering or smoothness; the multi-camera case was not tested and, given the resource results above, is identified as the variable most likely to reveal a scaling limit.

D. Reference Comparison with RViz2

As a reference point, RViz2 run on the robot's own onboard compute (Jetson-class hardware) used $\sim 58.6\%$ CPU and ~ 215 MB resident memory, substantially higher than the dashboard's browser-tab footprint above. This is not a controlled, same-machine comparison -- RViz2 competed with other onboard processes rather than running on the operator's laptop -- but is

directionally consistent with this paper's motivation for offloading visualization off constrained hardware.

E. Summary and Functional Outcomes

Table I summarizes all eight tests. A consistent pattern holds throughout: outbound control is lightweight and network-robust, while inbound visualization is bandwidth- and latency-sensitive, and resource cost is driven mainly by continuous `/tf` processing rather than rendering or control logic. All figures were obtained through manual timing and built-in tools across single-session trials, not automated or statistically repeated measurements.

TABLE I. SUMMARY OF INFORMAL EVALUATION RESULTS

Test	Key Result
1. Connection & reliability	~ 5.0 s connect; ~ 1.0 s stop/reconnect; < 1 s e-stop
2. Message latency	<code>/scan</code> ~ 10.2 Hz, <code>/tf</code> ~ 190 Hz; ~ 3 – 4 s visualization delay; control immediate
3. Render/draw rate	~ 10 – 15 FPS observed (60 FPS ceiling)
4. CPU & memory	~ 14 – 19% CPU, ~ 89 MB memory (idle approx active)
5. Network bandwidth	~ 150 – 250 Kbps out; ~ 10 – 35 MB in (> 5 min)
6. Teleop responsiveness	< 1 s latency, unaffected by network delay
7. Scalability sweep	No difference across map size/LiDAR model; multi-camera untested
8. RViz2 comparison	58.6% CPU, ~ 215 MB (on Jetson, not laptop; uncontrolled)

X. COMPARISON WITH EXISTING DASHBOARDS

Tables II–IV compare SDV Dashboard against the systems reviewed in Section II along architecture, capability, and deployment; cell contents are drawn from cited documentation and direct inspection of the dashboard's source code.

TABLE II. ARCHITECTURE COMPARISON

System	Architecture	Backend	Deployment
SDV Dashboard	Thin client	<code>rosbridge</code> + <code>web_video</code>	Open HTML file
RViz2 [2], [3]	Native desktop	<code>rclcpp</code> / <code>rclpy</code>	ROS 2 install
Webviz [7]	Browser SPA	<code>rosbridge</code>	Hosted/self-hosted
Foxglove [8], [9]	Desktop + Web	<code>foxglove_bridge</code>	Account required
ROSboard [10], [12]	Web server	Tornado WebSocket	Python package

TABLE III. CAPABILITY COMPARISON

System	Control	3D View	Mobile
SDV Dashboard	Teleop + Nav2	2D only	Yes
RViz2	Via panels	Full 3D	No
Webviz	Limited	WebGL 3D	Browser support
Foxglove	Twist publish	Full 3D	Browser-based
ROSboard	Limited [12]	Basic 2D	Yes [12]

TABLE IV. DEPLOYMENT COMPARISON

System	Robot Install	Use Case
SDV Dashboard	rosbridge + web_video	Student robots, teleop
RViz2	Standard ROS 2	Debugging, 3D visualization
Webviz	rosbridge or bag files	Bag analysis
Foxglove	foxglove_bridge	Multi-robot monitoring
ROSboard	Python package	Mobile robot monitoring [12]

SDV Dashboard and Webviz share a core dependency on `rosbridge_server`, differing mainly in that Webviz is a configurable, panel-based application while this dashboard is fixed-purpose. Foxglove and ROSboard both move away from `rosbridge_suite` -- Foxglove toward its own bridge for performance [8], ROSboard toward a custom protocol for compatibility and security [10] -- while RViz2 alone offers full native 3D rendering, a capability the browser-based alternatives forgo for portability. SDV Dashboard's positioning is as a zero-installation, zero-account, single-robot operator console rather than a fleet-management or data-inspection platform.

XI. CONCLUSION

This paper presented SDV Dashboard, a single-file, browser-based monitoring and teleoperation interface for ROS 2 mobile robots built on `rosbridge_suite` and `roslibjs`, including its transform-composition pose estimation, offscreen-canvas rasterization strategy, and safety and security considerations, positioned relative to RViz2, Webviz, Foxglove, ROSboard, and browser-native ROS 2 environments along architecture, capability, and deployment dimensions. An informal evaluation found the dashboard's control path lightweight and responsive regardless of network conditions, with a browser-tab footprint (~14--19% CPU, ~89 MB) substantially below an uncontrolled reference measurement of RViz2 on the robot's own onboard compute (~58.6% CPU, ~215 MB); a rigorous, same-machine comparison and a multi-camera scalability test remain future work. The dashboard's contribution is architectural rather than algorithmic: by depending only on already-standard `rosbridge_suite` and video-streaming services and contributing no backend code of its own, it lets an operator monitor and drive a ROS 2 robot with nothing more than a web browser.

REFERENCES

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, May 2022.
- [2] Open Robotics, "rviz2," ROS 2 Jazzy package documentation. [Online]. Available: <https://docs.ros.org/en/jazzy/p/rviz2/>
- [3] ros2/rviz GitHub repository, "ROS 3D Robot Visualizer." [Online]. Available: <https://github.com/ros2/rviz>
- [4] C. Crick, G. Jay, S. Osentoski, B. Pitzer, and O. C. Jenkins, "ROS and rosbridge: Roboticians out of the loop," in *Proc. 7th ACM/IEEE Int. Conf. Human-Robot Interaction (HRI)*, Boston, MA, USA, 2012, pp. 493--494.
- [5] RobotWebTools, "rosbridge_suite: Server implementations of the rosbridge v2 protocol," GitHub repository, `ros2` branch. [Online]. Available: https://github.com/RobotWebTools/rosbridge_suite
- [6] RobotWebTools, "roslibjs: The standard ROS JavaScript library," GitHub repository. [Online]. Available: <https://github.com/RobotWebTools/roslibjs>
- [7] Cruise Automation, "webviz: web-based visualization libraries," GitHub repository README. [Online]. Available: <https://github.com/cruise-automation/webviz>
- [8] Foxglove, "Using rosbridge with ROS 2," Foxglove Blog. [Online]. Available: <https://foxglove.dev/blog/using-rosbridge-with-ros2>
- [9] Foxglove, "Foxglove 2.0 --- integrated UI, new pricing, and open source changes," *Open Robotics Discourse*, Mar. 2024. [Online]. Available: <https://discourse.openrobotics.org/t/foxglove-2-0-integrated-ui-new-pricing-and-open-source-changes/36583>
- [10] D. Venkatraman, "rosboard: ROS node that turns your robot into a web server to visualize ROS topics," GitHub repository README. [Online]. Available: <https://github.com/dheera/rosboard>
- [11] Foxglove, "Pricing," Foxglove product website. [Online]. Available: <https://foxglove.dev/pricing>
- [12] D. Venkatraman, "Introducing ROSboard: Web-based visualizations for ROS1 and ROS2," *ROS General*, *Open Robotics Discourse*, Jul. 2021. [Online]. Available: <https://discourse.openrobotics.org/t/introducing-rosboard-web-based-visualizations-for-ros1-and-ros2/21311>
- [13] T. Fischer, I. Paredes, M. Batchelor, T. Beier, J. Haviland, S. Traversaro, W. Vollprecht, M. Schmitz, and M. Milford, "ROS2WASM: Bringing the Robot Operating System to the web," *arXiv preprint arXiv:2409.09941*, 2024.
- [14] tf2 documentation contributors, "Introduction to tf2," ROS 2 documentation. [Online]. Available: <https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Tf2/Introduction-To-Tf2.html>